date.timezone .ini config directive, and when DateTime objects are created with a date/time but no time zone, they will either use the default or a fixed zone, depending on whether the constructor argument is a timestamp or a formatted date/time string. Here's what I mean:

```
var_dump(date_default_timezone_get());
$dt = new DateTime('@1725148800');
print $dt->format('T') . PHP_EOL;

$dt = new DateTime('2024-09-01');
print $dt->format('T');

Output:
string(3) "UTC"
GMT+0000
UTC
```

The code above demonstrates a few things. Firstly, the date.timezone (default PHP time zone) is UTC, which we see from the output of date_default_timezone_get(). Next we instantiate a DateTime object with a Unix timestamp, which ignores the default timezone (as it should, because Unix timestamps are always UTC) and will always set the object's zone to GMT+0000. Finally, objects created with a formatted string will get PHP's default (UTC in our case). We've just had our first glimpse of one of the functions provided by the date module; we'll be seeing more of those in the next section.

Modifying DateTime

We've already seen DateTime::modify(), but there are other methods available for modifying data. In all cases a DateTime instance will be modified and return a reference to itself, and a DateTimeImmutable will return a new object of the same class. The methods are as follows: